

MBE 뉴런을 MLP 에 적용

데이터 생성

가우시안 블롭(Gaussian Blobs) 형태의 합성 데이터(Synthetic Data) 생성 기법을 사용합니다.

```
1  import torch
2  import matplotlib.pyplot as plt
3  from sklearn.datasets import make_blobs
4  from torch.utils.data import TensorDataset, DataLoader
5
6  # 1. 3-Class 노이즈 데이터 생성
7  # n_samples: 데이터 개수, n_features: 입력 차원(MLP input size), centers: 클
   래스 개수(3), cluster_std: 노이즈(퍼짐) 정도
8  X_np, y_np = make_blobs(n_samples=3000, n_features=20, centers=3,
   cluster_std=2.0, random_state=42)
9
10 # 2. PyTorch Tensor로 변환
11 X_tensor = torch.tensor(X_np, dtype=torch.float32)
12 y_tensor = torch.tensor(y_np, dtype=torch.long)
13
14 # 3. DataLoader 구성 (Train/Test 분리)
15 dataset = TensorDataset(X_tensor, y_tensor)
16 train_size = int(0.8 * len(dataset))
17 test_size = len(dataset) - train_size
18 train_dataset, test_dataset = torch.utils.data.random_split(dataset,
   [train_size, test_size])
19
20 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
21 test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
22
23 print(f"입력 데이터 형태: {X_tensor.shape}") # [3000, 20]
24 print(f"타겟 데이터 형태: {y_tensor.shape}") # [3000]
```

다 차원 n class 데이터를 생성합니다.

MLP 구조(예시)

다 차원 입력(Input)을 받아 n개의 클래스(Output)로 분류하는 Toy MLP를 만들 때, 가장 중요한 점은 "이 MLP가 나중에 우리가 SNN으로 변환할 타겟인 트랜스포머(TinyLLaVA)의 축소판 역할을 해야 한다"는 것입니다.

따라서 단순히 Linear 층만 쌓는 것이 아니라, 논문에서 SNN으로 변환하겠다고 선언한 핵심 비선형 요소들인 **LayerNorm** 과 **GELU** 를 반드시 포함시켜야 합니다.

```

1  import torch
2  import torch.nn as nn
3
4  class ToyTransformerMLP(nn.Module):
5      def __init__(self, input_dim=20, hidden_dim=64, num_classes=3):
6          super(ToyTransformerMLP, self).__init__()
7
8          # 첫 번째 블록 (트랜스포머의 FFN 구조 모방)
9          self.fc1 = nn.Linear(input_dim, hidden_dim)
10         self.ln1 = nn.LayerNorm(hidden_dim) # MBE 변환 타겟 1
11         self.gelu1 = nn.GELU()             # MBE 변환 타겟 2
12
13         # 두 번째 블록
14         self.fc2 = nn.Linear(hidden_dim, hidden_dim)
15         self.ln2 = nn.LayerNorm(hidden_dim) # MBE 변환 타겟 1
16         self.gelu2 = nn.GELU()             # MBE 변환 타겟 2
17
18         # 출력단
19         self.fc3 = nn.Linear(hidden_dim, num_classes)
20
21         # 참고: 학습 시에는 CrossEntropyLoss를 쓰기 때문에 Softmax를 빼두지만,
22         # 추론(Inference) 단계에서 MBE_Softmax 교체 테스트를 위해 명시적으로 사
23         self.softmax = nn.Softmax(dim=-1)   # MBE 변환 타겟 3
24
25     def forward(self, x):
26         x = self.fc1(x)
27         x = self.ln1(x)
28         x = self.gelu1(x)
29
30         x = self.fc2(x)
31         x = self.ln2(x)
32         x = self.gelu2(x)
33
34         logits = self.fc3(x)
35         return logits
36
37     def predict_probs(self, x):
38         # SNN 변환 후 Softmax 근사 성능을 평가하기 위한 별도 함수
39         logits = self.forward(x)
40         return self.softmax(logits)
41
42 # 모델 생성 확인
43 model = ToyTransformerMLP()
44 print(model)

```

2. 실험(검증) 진행 시나리오

이 구조를 만들었다면, 다음과 같은 순서로 연구 검증을 진행하시면 됩니다.

Step 1. ANN 정상 학습 (Baseline) 앞서 만든 노이즈 데이터(`make_blobs`)를 이 모델에 넣고 Adam 옵티마이저와 CrossEntropyLoss를 이용해 학습시킵니다. (정확도 95% 이상 도달 목표)

Step 2. 데이터 분포 수집 (Calibration - 논문의 핵심!) 학습이 끝난 후, 모델을 `eval()` 모드로 두고 훈련 데이터를 한 번 더 흘려보냅니다. 이때 `ln1`, `gelu1`, `ln2`, `gelu2`에 들어가는 입력값(텐서)의 최소/최대 범위(Interval)를 기록합니다.

- **이유:** MBE 뉴런은 무한한 범위를 커버하지 못하므로, 실제 데이터가 분포하는 범위(예: -5.0 ~ 5.0) 내에서만 오프라인으로 0과 1의 스파이크 강도를 피팅(Fitting)해야 하기 때문입니다.

Step 3. Training-Free SNN 변환 (Swap) 학습된 `ToyTransformerMLP` 에서 부동 소수점 곱셈, 비선형 활성화 함수 등을 직접 구현한 MBE 근사 모듈로 교체합니다. 이때 MBE 모듈은 수집한 데이터 분포에서 오버피팅된 모델을 사용합니다.

Step 4. 성능 및 효율성 평가

SNN으로 교체된 모델에 테스트 데이터를 타임스텝 T (예: 16 또는 32) 동안 흘려보냅니다.

1. **정확도:** ANN의 95% 정확도가 SNN에서도 94~95%로 '거의 손실 없이(Near-lossless)' 유지되는지 확인합니다.
2. **에너지 효율:** 각 타임스텝에서 발생한 스파이크의 개수(Firing Rate)를 세어, 기존 ANN의 부동소수점 곱셈(MAC) 대비 SNN의 덧셈 연산(SOPs)이 얼마나 전력을 아꼈는지 계산하여 그래프로 뽑아냅니다.